An Efficient NDN Routing Mechanism Design in P4 Environment

Xingchang Guo*, Ningchun Liu*, Xindi Hou*, Shuai Gao*†, Huachun Zhou*
*School of Electronic and Information Engineering, Beijing Jiaotong University, China
†PCL Research Center of Networks and Communications, Peng Cheng Laboratory, Shenzhen, China
{xchguo, nchliu, Freezerburn, shgao, hchzhou}@bjtu.edu.cn

2021 2nd Information Communication Technologies Conference (ICTC) | 978-1-6654-0474-7/20/$31.00 ©2021 IEEE | DOI: 10.1109/ICTC51749.2021.9441639

***Abstract*—Name Data Networking (NDN) is a clean-slate network redesign that uses content names for routing and addressing. Facing the fact that TCP/IP is deeply entrenched in the current Internet architecture, NDN has made slow progress in industrial promotion. Meanwhile, new architectures represented by SDN, P4, etc., provide a flexible and programmable approach to network research. As a result, a centralized NDN routing mechanism is needed in the scenario for network integration between NDN and TCP/IP. Combining the NLSR protocol and the P4 environment, we introduce an efficient NDN routing mechanism that offers extensible NDN routing services (e.g., resources-location management and routing calculation) which can be programmed in the control plane. More precisely, the proposed mechanism allows the programmable switches to transmit NLSR packets to the control plane with the extended data plane. The NDN routing services are provided by control plane application which framework bases on resource-location mapping to achieve part of the NLSR mechanism. Experimental results show that the proposed mechanism can reduce the number of routing packets significantly, and introduce a slight overhead in the controller compared with NLSR simulation.**

***Keywords*—NDN, P4, routing mechanism**

I. Introduction

The Internet gradually shows conflicts with its original design, such as the location binding of TCP/IP network resources and difficulties in supporting mobility. To solve these problems, numerous clean slate redesigns of the Internet are proposed by academics, and NDN [1]is one of the most representative. However, achieving the rollout of these new network architectures is a slow and challenging process. Networking research is driven by two facts [2]: (i) the current Internet architecture has inherent flaws and alternatives should be explored and (ii) there is also the fact that the TCP/IP is deeply entrenched and difficult to change in current network. Therefore, the research of heterogeneous network integration [3] [4]that can be backward compatible with new network architectures has become a new hot topic.

In recent years, the booming development of SDN [5]and P4 [6]has provided new avenues for network research. NDN's network layer stateful forwarding makes it a natural multicast and in-network ubiquitous caching feature [1]. Hence, achieving the integration of TCP/IP and NDN can balance compatibility with the advantages of NDN, which has become one of the main research orientation in academia [7]. Currently, two main types of technology lines are used for TCP/IP and NDN integration: (i) by dual-stack routers/gateways [8]or (ii) based on protocol-independent forwarding [9] [10]. The former is difficult to modify after deployment and does not have the flexibility for other identification spaces. On the contrary, the latter could realize coexistence and integration of heterogeneous networks and achieve the main functions of networks based on P4 for IP, NDN, etc. Because of better scalability, it becomes a new direction of heterogeneous network integration research. With this idea, apart from some research on the data plane [10]–[12], there is also a clear lack of NDN routing architecture under centralized control.

The NDN project team proposed OSPFN [13] and NLSR [14] [15], and related scholars have implemented the SD-NDN routing scheme through OpenFlow [16] [17]. However, the distributed and stateful protocol runs counter to the centralized design concept of SDN, and the closed design of OpenFlow makes it difficult to promote in further research. If a centralized routing mechanism can be designed on the control plane based on P4 environment, we can facilitate NDN network integration and prove that the NDN concept can truly bring benefits to the network.

The rest of the paper is structured as follows: Section II introduces the current related work. Section III presents the architectural design. Section IV illustrates the experimental results and analysis. Section V presents conclusion.

II. Related Work

A. NDN Routing Functionalities

The routing protocol of NDN needs to provide multiple next hops to support multi-path forwarding, and the NDN project team has proposed two routing protocols based on link-state: OSPFN and NLSR. OSPFN is a routing protocol based on content names and defines nolsa as a new link state advertisement. Since the OSPF is used to generate FIB entries, the OSPFN still has many limitations, such as using the IP address as the NDN routerID and using tunnel encapsulation to cross the TCP/IP network, which limits the original advantages of NDN.

To overcome the shortcomings of OSPFN, NLSR uses Interest and Data packets to transfer the name prefix and link connection information between nodes, and content name instead of IP address is used to distinguish routers and links. Whenever a link failure/recovery or new neighbor discovery is

TABLE I
COMPARISON OF SDN BASED NDN ARCHITECTURES

Schemes	*Purpose*	*Data Plane*	*Open Issues*	*Year*
SRSC [16]	Stateless delivery by attaching routing addresses to NDN packets	OpenFlow	Difficulty in date plane programmability	2015
NDNFlow [17]	Prioritize NDN flows and assign them to queues of different devices in the network	OpenFlow	Difficulty in date plane programmability	2015
NDN.P4 [11]	P4 implementation of NDN switch and support a stateful variable length protocol	P4	Few focus on control plane	2016
ENDN [10]	Enable direct control of applications on forwarding strategies	Enhanced P4	Difficulty in supporting dynamic updates due to the pre-configured namespace	2020

detected by the NLSR, it generates a new LSA which prefix could be /<network>/NLSR/LSA/<version> and spreads it to the entire network. Fig. 1 shows the LSA format [14]. An interest packet which contains LSAs needs to be forwarded to all neighbors unless its copy has already existed in content store (CS). Otherwise, the packet will be forwarded until it is extended to every router.

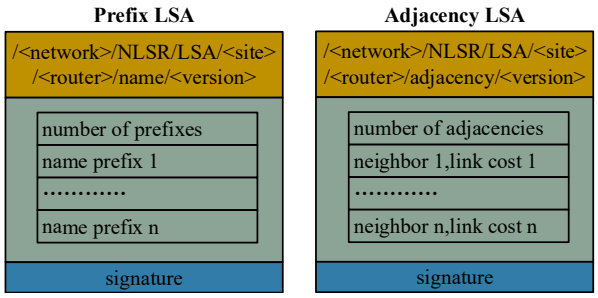

Fig. 1. LSA format.

Compared with the IP-based routing protocol OSPFN, NLSR has built-in routing update certification and capability on multipath forwarding as well as security by using the NDN native packet signature mechanism. However, a large number of broadcast Interest packets which contains LSAs will be generated during the process of routing convergence in the entire network [14]. NLSR uses NDN's distributed synchronization protocol ChronoSync [18] in its implementation, and exchanges the hash value of LSDB by root advise interest packets (RA Interest) and root advise data packets (RA Data) [14]. This method reduces the size of synchronization packets, but it still generates a large number of packets to meet timing synchronization requirement. NDN routers have to spend a large amount of resources to process these packets. Therefore, centralizing the distributed routing process to the controller has high research value in NDN, and it's performance on reducing the number of routing packets in IP network has been shown in [19].

B. SDN Based NDN

SDN controllers is designed for routing, caching, and distributing content based on user's requests, so as to reduce the complexity of NDN deployments and make the network more flexible in scheduling cache resources and interest packet paths. Hence, it has prompted scholars to explore the integration of SDN and NDN.

Some efforts have been proposed in the literature to support forwarding strategy in NDN. Elian Aubry et al. [16] presented NDN routing schemes SRSC. By establishing the forwarding rules for the NDN node, SRSC generates the FIB entry of the nearest content provider. Adrichem et al. [17] challenged the traffic scheduling for NDN flows. They prioritize NDN flows and assign them into the queues of different devices in the network. Due to the use of OpenFlow in the data plane, related researches have poor data-plane programmability.

Rui Miguel et al. [11] proposed a P4 implementation of NDN switch which focuses on supporting a stateful variable length protocol, but did not involve the generation of flow entries. To solve the routing problem, Ouassim Karrakchou et al. [10] proposed an enhanced NDN architecture with a P4-programmable data plane. By linking pre-configured namespace with a set of associated services, the control plane generates the corresponding data plane configuration. Since the namespace is pre-configured, the control plane cannot dynamically update the locations of the content producers.

Table I shows the comparison of existing SD-NDN architectures. In summary, while many approaches focus on the architectural design of routing strategies in NDN, few solves the problem of routing protocols. Moreover, while packet control of programmable data plane provides a new tool for network research, there is still a gap on the compatibility for traditional NDN routing protocols.

III. Proposed Architecture

A. Architecture Overview

Following the principles of SDN, this architecture can be separated into control plane and data plane. The main goal of the control plane is to establish resource-location mapping by processing NLSR routing packets. According to the request of Interest packets, these services are then converted into data plane configuration and installed in the P4 switch.

On the data plane, there are P4 switches, consumers and producers running the NLSR protocol, and packets will match one or more tables with the order defined by the P4 program. Fig. 2 shows an overview of the architecture and indicates the trend of different types of packets when a new content prefix is generated.

In addition to neighbor discovery and link recovery, NLSR routing packets mainly include four types: RA Interest (served as timed routing synchronization request), RA Data (as response), LSAs Interest (served as routing update request) and LSAs Data(as response). In the architecture of SDN, the control link between the controller and switches is relatively

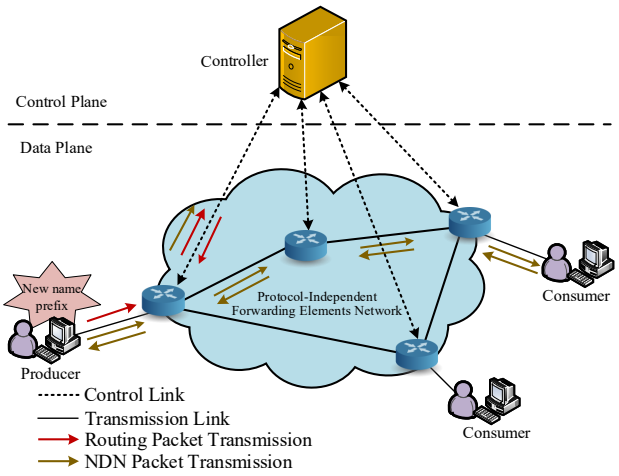

Fig. 2. The architecture overview.

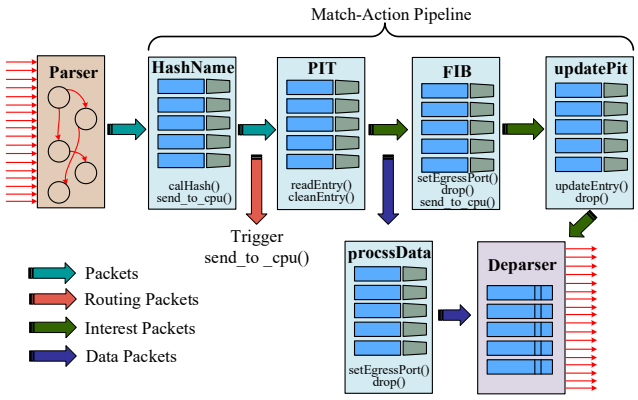

Fig. 3. The tables and actions in pipeline.

reliable [19], and the packet loss rate of the control channel is acceptable. The routing message interaction only occurs between the NDN node which runs NLSR process and the P4 switches, and all the routing processes are carried out within the controller. Since the controller has a global topology, it has no need to confirm the location of the opposite node like the traditional NDN network, hence, the RA Data packets are not used in this mechanism.

B. Data Plane

P4 currently has relatively poor support for strings [20], in order to match name prefix, paper [11] suggest to use the hash value of name for the longest prefix matching or exact matching. In this mechanism, we have enhanced the data plane of [11] with the following improvements to enable interaction with the control plane. Fig. 3 shows the tables and actions in pipelines.

- We add action send_to_cpu(), which is used to forward Interest packets that fail to hit any table entry to the controller through P4runtime.
- We modify the prefix of NLSR matching process in the pipeline, sending the NLSR packets to controller in the form of Packet_in.
- We add a Packet_out message processing, decapsulating the routing calculation results in metadata and NLSR routing packets from the controller, and forwarding them to the corresponding port.

The parser in the P4 program will only process the NDN type packets in an Ethernet frame. If the ingress port of the packet is CPU_PORT, which indicates that the packet is a Packet_out packet from the controller and what is encapsulated is an NLSR routing packet or a NACK packet. The P4 program reads the customized metadata of the packet and forwards it from the designated port according to the port information it carries.

If the ingress port is not CPU_PORT, it will apply *hashName*. This table calculates the hash value of the content name, including the hash value of each name component h(block1),h(block2), etc., and the hash value of the entire content name h(full_name). The former is used for longest prefix matching in *FIB*, and the latter is used for reading and writing the *PIT*'s registers. In this paper, the packet is said to be a NLSR routing packet while its first component is "NLSR", and it serves as a trigger for the action send_to_cpu().

Interest packet will apply *PIT* and *FIB*. If there is no matching entry, the send_to_cpu() is triggered. The Data packet will match *PIT*, in which the forwarding port in the register will be read using h(full_name) and the packet will be forwarded using action setEgressPort().

C. Control Plane Design

The P4 switches are connected to the controller through the control channel of P4runtime and forward the NLSR packets to the upper APP for routing processing. The specific APP framework on ONOS [21] is shown in Fig. 4, and the following is the design of each module.

1) Initializd module: To use ONOS's services, each APP must register in the ApplicationManager to obtain a unique APP-id and register the packet listening service. After the controller establishing a connection with the P4 switches, some initialization settings are required, mainly includes listener registration and setting some default flow entries such as matching LLDP packets used for topology discovery.

2) Resources-Location Mapping Database (RLM-DB): This module is used to update and query the location of the content and provides information for other modules. The controller is responsible for collecting the name prefix in LSAs Data and maintains a resource-location mapping in form of name prefix, connectpoint. The name prefix indicates the source name and the connectpiont presents the location of a host (producer) in ONOS. There are mainly two naming rules in this paper, one is used for application naming as the Interest name of LSA in NLSR using the prefix of /NLSR/LSA/<routerID>/<version>. The other is for synchronization using the name prefix of /NLSR/ChronoSync/<routerID>/<version>.

Query process: The Interest packet sent by the NDN node is addressed by the content name, and a flow entry is matched when the packet arrives at the P4 switch and sent out via the

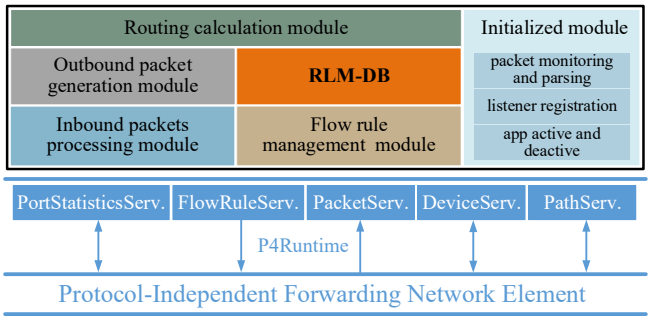

Fig. 4. ONOS control plane framework.

setEgressPort() which is parameterized by a port number on that switch. P4 switches maintains mapping pairs {name prefix hash, resource location port} in the form of flow entries. When the flow entries do not have the required mapping relationship, a query request needs to be initiated to the controller. The controller queries RLM-DB for the requested mapping pair, it is installed to the P4 switches along the path.

Update process: Producers will publish new content and also withdraw existing content. To accurately reflect the p-resence of resources, each mapping pair in the RLM-DB has a survival time TTL. In this paper, we perform the survival time update with reset. The TTL of each mapping pair in the RLM-DB decreases with time, and when there is no LSAs Data message about the name prefix within a timeout period, the mapping pair will be deleted. To reduce the name prefix matching time during updates, the controller maintains a version cache $\mathbb{S}$ to save the LSAs's version and routerID. The algorithm pseudocode of chronoSync packetes process is shown in **Algorithm 1**.

Using inbound packet processing module, the controller can parse out the routerID rid, hash value $hash_{rid}$ and version number ver_{rid} of the RA Interest. RLM-DB performs the name prefix hash calculation to calculate the current hash value $hash_c$ of stored name prefix. This module will check whether the node has been synchronized through the version store $\mathbb{S}$. The key-value pair of $\{rid, ver_{rid}\}$ is stored in $\mathbb{S}$. This module will use rid to retrieve in $\mathbb{S}$. If the corresponding value is found, the version number will be compared. In this paper, it is assumed that after performing a routing synchronization, the version number belongs to the name prefix of the RA Interest, will increase by 1, so as to distinguish the invalid message with a long transmission time due to delay in the network. If the version number is higher, it means that a new content name is declared in rid, outbound packets generation module is called to generate LSAs Interest packets to request rid. If the version number is lower, it means that the synchronization information is outdated and does not need to be synchronized. If the corresponding key-value pair cannot be found, router rid is synchronizd for the first time. The key-value pair $\{rid, ver_{rid}\}$ should be recorded in $\mathbb{S}$, and LSAs Interest is constructed for synchronization.

3) Inbound packet processing module: This module processes Packet_in, there are three types of packets:

Interests that failed to hit any FIB entry: This module will parse out the name of the interest and query it in RLM-DB. If the corresponding mapping pair is found, this module will call the routing calculation module to generate flow entries along the path between the location of resource and consumer. If not, it is considered that the interest packet cannot be satisfied in the network. The outbound packets generation module will be called to construct a NACK packet.

RA Interest: The hash value of the LSDB in RA Interest is parsed and compared with the hash value of RLM-DB to determine whether the name prefix needs to be updated.

LSAs Interest and LSAs Data: The controller needs to synchronize the routing information with the NLSR node in the network regularly by sending the RA Interest. When it is found that the hash value of the LSDB is different from the controller, the LSAs Interest is constructed for prefix update. The RLM-DB module will be called to construct an LSA Data packet ,and send the packet to the requester. In addition, if the controller receives the LSAs Data packet, name prefixes in it will be parsed to update RLM-DB.

4) Outbound packets generation module: This module is responsible for constructing NACK and NLSR routing packets. First the ByteBuffer object is created, which contains the serialization of the NACK or routing packet. It is added as a load to the Ethernet frame object instantiated in ONOS. The Ethernet frame object and TrafficTreatment object (containing the forwarding device and port of the packet) are encapsulated in the instantiated OutboundPacket object. Finally the packetService is called to send the OutboundPacket out.

Algorithm 1: ChronoSync packets process

input : rid: routerID in RA Interest name
$hash_{rid}$: hash value of RA Interest
ver_{rid}: version in RA Interest name
version store $\mathbb{S}$ of form $[\{rid_1, version_1\}, ...]$
output: whether to send LSAs for synchronization.

```
RLM-DB performs a hash calculation and obtains the current hash value hash_c;
use rid to retrieve in S ;
if corresponding version number is ver_c then
    if ver_rid > ver_c then
        if hash_rid = hash_c then
            do not sync;
        else
            sync;
        end
    else
        do not sync;
    end
else
    corresponding version number is not found;
    record {rid,hash_rid} in S, and sync;
end
```

5) FlowRule management module: This module is responsible for the interpretation, generation, and distribution of FlowRule. In ONOS, FlowRule is an object of flow entry

class, which is a high-level abstraction of flow entries with a set of match and action. ONOS is responsible for expressing the network command as multiple high-level flow rules, and the FlowRule subsystem manages and installs the flow rules into the infrastructure, including mapping the standard header-actions to the pipeconf defined by the P4 program.

6) Routing calculation module: In ONOS, a link represents a one-hop connection, and a path is an end-to-end connection. After querying RLM-DB, this module calculates the weight of each link along the path between requester and resource location. The PortStatisticsService could provide the current sending rate of the port (*link_load_speed*) and the working speed of the switch port (*link_port_speed*) can be obtained through DeviceService. Equation 1 is used to calculate weight of a link. This module performs path selection with the weighted Dijkstra algorithm. The smaller the above weights, the higher the priority in link selection. Finally, the corresponding FlowRules along the path or FlowObjective which is provided for other pipeline-agnostic applications is generated.

$$\begin{cases} \text{link\_load\_speed} = \max(\text{link\_load\_speed}_{\text{src}}, \text{link\_load\_speed}_{\text{dst}}) \\ \text{link\_port\_speed} = \min(\text{link\_port\_speed}_{\text{src}}, \text{link\_port\_speed}_{\text{dst}}) \\ \text{Weight} = W_{\max} - (\text{link\_load\_speed} - \text{link\_port\_speed}) * W_{\max}) \end{cases} \quad (1)$$

D. Interaction between Control Plane and Data Plane

Fig. 5 describes the interaction design between the data plane and the controller. As shown in the dotted line, the RA Interest are sent regularly to exchange the hash value of LSDB.

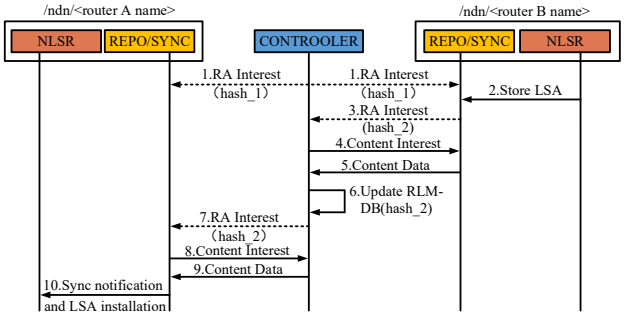

Fig. 5. Interactive sequence diagram.

The controller will periodically calculate the hash value of the content prefix data in the RLM-DB as $hash\_1$, and send the RA Interest to the NDN node running NLSR in the form of a Packet_out message(step 1). When routerB declares a new content prefix and writes it in Sync slice [14], routeB's Sync will create a new RA Interest with $hash\_2$, and send it to the controller through the action send_to_cpu() of the data plane(step 2 and step 3). After parsing the RA Interest and processing Algorithm1, the controller determines whether route message needs to be updated, and uses the outbound packets generation module to construct a LSA Interest packet to request the new LSA from routerB(step 4). Thecontroller parses the LSAs Data responsed by routerB, updates the RLM-DB and sends RA Interest with a new hash value of $hash\_2$ (step 5 to step 7). Since routerA receives RA Interest with a different hash value, it will send LSA Interest to the controller to request an routing update, and the controller will generate an LSA Data packet according to RLM-DB and sends Packet_out to RouterA(step 8 and step 9). The NLSR process of routeA performs a series of content synchronization until the LSDBs of routerA and B are consistent, then route synchronization is completed(step 10).

IV. Simulation Experiments and Analysis

A. Experimental Environment

This section evaluate the performance of centralized NDN routing architecture proposed in this paper in term of P4 switch processing time, messaging overhead and convergence time.

In the simulation, the NLSR nodes maintains a set of name prefixes as the LSDB, and uses the packet generator to construct and send various routing packets to the network. ONOS of version 2.2 is used as the controller and runs a custom app to handle routing packets. Controller is run on the device with Intel(R) Xeon(R) Silver 4214 CPU 2.20GHz processors and 64G of RAM. The data plane uses mininet version 2.3.0b to build the network topology with the bmv2 (version 1.13.0) as the switch. This node runs on a device with an Intel(R) Xeon(R) CPU X3440 @ 2.53GHz and 16G of RAM. The kernel versions of the above devices are Linux version 4.4.0-197-generic buildd@lcy01-amd64-026, Ubuntu 16.04.

B. Simulation Result

In order to verify the efficiency of the propsed mechanism, we perform simulation of traditional NLSR (*NLSR-Sim*) and the proposed mechanism (*NLSR-P4*) for comparison and evaluate the average number of NDN routing packets. The same topology and interval (10s to 30s) of RA Interest message in [14] is set to perform the comparison.

The average packets overhead is depicted in Fig. 6. As the RA Interests are integrated in the controller in a form of Packet_in message rather than a wide range broadcast, it can be seen that the number of RA Interest occurs a sharp decrease by about 42.9Meanwhile, the LSAs Interest and the LSAs Data also show maximum decreases of 33.4% and 30.4%. Since the controller maintains the RLM-DB in the network, it speeds up the synchronization of node LSDBs and changes the mechanism of mutual updates between two nodes in traditional NLSR, which leads to reduce of update times.

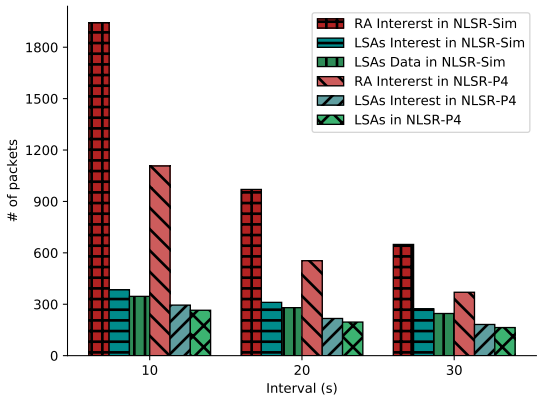

Fig. 6. Number of packets sent and received.

TABLE II
COMPARISONS OF ROUTE PERFORMANCE BETWEEN TRADITIONAL NLSR (NLSR-SIM) AND THE PROPOSED MECHANISM (NLSR-P4)

Schemes	Average number of packets and resource occupancy			Average convergence time percentage(%)
	RA Intereset	LSAs Interest	LSAs Data	
NLSR-Sim	1942	384	346	31.3
NLSR-P4	1107	295	265	19.3

Convergence time is the interval required for all LSDB to reach agreement. We sampled the number of various types of packets from the beginning to 300 seconds, as shown in the Fig. 7. There are no new name prefixes are declared in the first 150 seconds, and the average precentage of convergence time is obtained by (convergence time)/150s and is used to reflect the rapidity of LSDB synchronization. The average time percentage in the NLSR-Sim is 31.3%, proving that the convergence time is longer than the NLSR-P4. After 150 seconds, new name prefixes are generated and the number of packets in the NLSR-P4 is much smaller and climbs slowly. Results show that it is of better effient to fufill the LSDB synchronization through controller than traditional broadcast. Table. II shows the comparisons of routing performance between NLSR-Sim and the NLSR-P4.

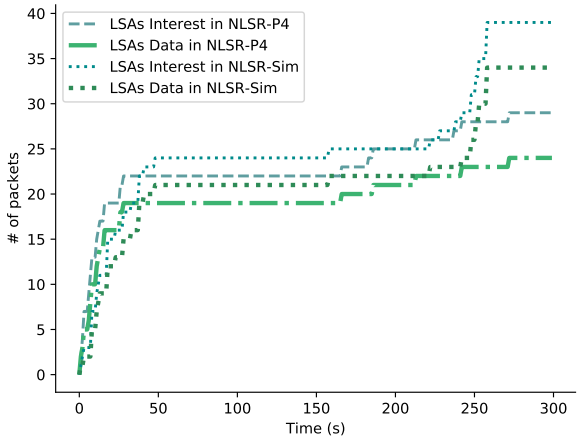

Fig. 7. Routing packet count.

V. CONCLUSIONS

In this paper, we propose a NDN routing mechanism in P4 environment to realize resource-location mapping management in heterogeneous network integration. The data plane of existing researches is expanded to support interaction with the control plane, and a routing packets processing APP framework is proposed with the function of each modules. The simulation is carried out to evaluate the feasibility and performance improvement. The result shows that the number of routed packets is reduced up to 42.9% and the routing convergence time is decreased by 12%.

ACKNOWLEDGMENT

This work was supported by the National Key Research and Development Program of China (No. 2019YFB1802503), the National Nature Science Foundation of China (No. 61972026, No. 61802014) and the project "FANet: PCL Future Greater-Bay Area Network Facilities for Large-scale Experiments and Applications (No. LZC0019)".

REFERENCES

[1] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, k. claffy, P. Crowley, C. Papadopoulos, L. Wang, and B. Zhang, "Named data networking," *SIGCOMM Comput. Commun. Rev.*, vol. 44, p. 6673, July 2014.

[2] J. McCauley, Y. Harchol, A. Panda, B. Raghavan, and S. Shenker, "Enabling a permanent revolution in internet architecture," in *Proceedings of the ACM Special Interest Group on Data Communication*, SIGCOMM '19, (New York, NY, USA), p. 114, Association for Computing Machinery, 2019.

[3] Y. Hu, D. Li, P. Sun, P. Yi, and J. Wu, "Polymorphic smart network: An open, flexible and universal architecture for future heterogeneous networks," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 2515–2525, 2020.

[4] J. Wu, "Thoughts on the development of novel network technology," *Science China Information Sciences*, vol. 61, no. 10, p. 101301, 2018.

[5] D. Kreutz, F. M. V. Ramos, P. E. Verssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[6] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *SIGCOMM Comput. Commun. Rev.*, vol. 44, p. 8795, July 2014.

[7] S. Luo, S. Zhong, and K. Lei, "Ip/ndn: A multi-level translation and migration mechanism," in *NOMS 2018 - 2018 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–5, 2018.

[8] T. Refaei, J. Ma, S. Ha, and S. Liu, "Integrating ip and ndn through an extensible ip-ndn gateway," in *Proceedings of the 4th ACM Conference on Information-Centric Networking*, ICN '17, (New York, NY, USA), p. 224225, Association for Computing Machinery, 2017.

[9] W. Feng, X. Tan, and Y. Jin, "Implementing icn over p4 in http scenario," in *2019 2nd International Conference on Hot Information-Centric Networking (HotICN)*, pp. 37–43, 2019.

[10] O. Karrakchou, N. Samaan, and A. Karmouch, "Endn: An enhanced ndn architecture with a p4-programmabie data plane," in *Proceedings of the 7th ACM Conference on Information-Centric Networking*, ICN '20, (New York, NY, USA), p. 111, Association for Computing Machinery, 2020.

[11] S. Signorello, R. State, J. Franois, and O. Festor, "Ndn.p4: Programming information-centric data-planes," in *2016 IEEE NetSoft Conference and Workshops (NetSoft)*, pp. 384–389, 2016.

[12] R. Miguel, S. Signorello, and F. M. V. Ramos, "Named data networking with programmable switches," in *2018 IEEE 26th International Conference on Network Protocols (ICNP)*, pp. 400–405, 2018.

[13] L. Wang, A. Hoque, C. Yi, A. Alyyan, and B. Zhang, "Ospfn: An ospf based routing protocol for named data networking," tech. rep., Technical Report NDN-0003, 2012.

[14] A. K. M. M. Hoque, S. O. Amin, A. Alyyan, B. Zhang, L. Zhang, and L. Wang, "Nlsr: Named-data link state routing protocol," in *Proceedings of the 3rd ACM SIGCOMM Workshop on Information-Centric Networking*, ICN '13, (New York, NY, USA), p. 1520, Association for Computing Machinery, 2013.

[15] L. Wang, V. Lehman, A. K. M. Mahmudul Hoque, B. Zhang, Y. Yu, and L. Zhang, "A secure link state routing protocol for ndn," *IEEE Access*, vol. 6, pp. 10470–10482, 2018.

[16] E. Aubry, T. Silverston, and I. Chrisment, "Srsc: Sdn-based routing scheme for ccn," in *Proceedings of the 2015 1st IEEE Conference on Network Softwarization (NetSoft)*, pp. 1–5, 2015.

[17] N. L. M. van Adrichem and F. A. Kuipers, "Ndnflow: Software-defined named data networking," in *Proceedings of the 2015 1st IEEE Conference on Network Softwarization (NetSoft)*, pp. 1–5, 2015.

[18] Z. Zhu and A. Afanasyev, "Let's chronosync: Decentralized dataset state synchronization in named data networking," in *2013 21st IEEE International Conference on Network Protocols (ICNP)*, pp. 1–10, 2013.

[19] H. Zhang and J. Yan, "Performance of sdn routing in comparison with legacy routing protocols," in *2015 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery*, pp. 491–494, 2015.

[20] "P4_16 Language Specification version 1.0.0." [EB/OL]. https://p4.org/p4-spec/docs/P4-16-v1.0.0-spec.html., Updated: December 16, 2020.

[21] "ONOS - Wiki. (2020)." [EB/OL]. https://wiki.onosproject.org/, Updated: December 16, 2020.